

Panduan Lengkap Enhanced Email Scraper Extension

1. PENDAHULUAN

1.1 Tujuan Project

Meningkatkan kapabilitas extension scraping email potensi dari berbagai sumber dengan fitur-fitur canggih berdasarkan arsitektur yang sudah ada.

1.2 Arsitektur Existing

```
text
ExtensionSwakarsa/
├── sections/
│   └── contact-scraper.html      # UI scraping contact
├── js/
│   └── contact_scraper.js       # Logic scraping utama
├── css/
│   ├── dashboard.css
│   ├── sidebar.css
│   └── style.css
└── manifest.json
```

2. SETUP AWAL

2.1 Clone Repository

```
bash
git clone https://github.com/Rmyonathan/ExtensionSwakarsa.git
cd ExtensionSwakarsa
```

2.2 Load Extension di Chrome

1. Buka Chrome → `chrome://extensions/`

2. Aktifkan Developer Mode (toggle kanan atas)
3. Klik "Load unpacked"
4. Pilih folder ExtensionSwakarsa
5. Extension akan muncul dengan icon yang ada

2.3 Struktur File yang Akan Dimodifikasi

text

```
/sections/contact-scraper.html      # UI enhancement
```

```
/js/contact_scraper.js              # Core logic enhancement
```

3. FLOW DAN FUNGSIONALITAS EXISTING

3.1 Current Flow

text

User Input → Start Scraping → Google Search → Email Extraction → Display Results

3.2 Fungsi-fungsi Kunci Existing:

contact_scraper.js:

1. `startContactScraping(config)` - Mulai scraping
2. `scrapeBingForEmails(source, keywords, dateRange, location)` - Scrape via Bing
3. `scrapeBingPage()` - Ekstrak halaman per halaman
4. `extractEmailsFromPage()` - Regex extraction email
5. `stopContactScraping()` - Stop proses

contact-scraper.html:

- Form input: keywords, date range, location
- Start/Stop buttons
- Progress bar
- Results display grid

4. PERUBAHAN YANG DIINGINKAN

4.1 Input Field Baru (contact-scraper.html)

```
html
<!-- Role Input -->
<div class="form-group">
  <label for="contact-role">Role/Category</label>
  <input type="text" id="contact-role" placeholder="e.g., HR, CEO, Recruiter, Manager">
</div>

<!-- Multiple Keywords Input -->
<div class="form-group">
  <label for="contact-keywords">Keywords (comma separated)</label>
  <input type="text" id="contact-keywords" placeholder="e.g., Machine Learning, Fashion, E-commerce, Startup">
  <small>Pisahkan dengan koma untuk multiple keywords</small>
</div>
```

4.2 Enhanced Email Patterns (contact_scraper.js)

```
javascript
// GANTI regex yang ada:
const emailPatterns = [
  /[a-zA-Z0-9._%+-]+@gmail\.com/gi,          // @gmail.com
  /info@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,     // info@domain
  /careers?@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi, // career(s)@domain
  /hr@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,       // hr@domain
  /recruitment@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi, // recruitment@domain
  /contact@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,  // contact@domain
  /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi // All emails
];
```

4.3 Results Management UI (contact-scraper.html)

```
html
<!-- Aksi Bulk Results -->
```

```

<div class="results-actions">
  <button id="select-all" class="btn">Select All</button>
  <button id="deselect-all" class="btn">Deselect All</button>
  <button id="delete-selected" class="btn btn-danger">Delete
Selected</button>
  <button id="edit-selected" class="btn">Edit Selected</button>
</div>

<!-- Email Actions -->
<div class="email-actions">
  <button id="copy-emails" class="btn btn-success">Copy Email
List</button>
  <button id="fast-blast" class="btn btn-primary">Fast Blast
(Gmail)</button>
  <button id="personalized-email" class="btn btn-warning">Personalized
Email (GAS)</button>
</div>

<!-- Results Table dengan Checkbox -->
<div class="results-table-container">
  <table id="contacts-table">
    <thead>
      <tr>
        <th width="30"><input type="checkbox"
id="select-all-checkbox"></th>
        <th width="200">Email</th>
        <th width="100">Role</th>
        <th width="150">Source</th>
        <th width="150">Keyword</th>
        <th width="200">Overview</th>
        <th width="100">Actions</th>
      </tr>
    </thead>
    <tbody id="contacts-table-body">
      <!-- Dynamic rows will be inserted here -->
    </tbody>
  </table>
</div>

```

5. IMPLEMENTASI KODING

5.1 Step 1: Modifikasi contact-scraper.html

TAMBAHKAN di bagian controls:

html

```
<!-- ROLE INPUT -->
<div class="form-group">
  <label for="contact-role">Role/Category</label>
  <input type="text" id="contact-role" placeholder="e.g., HR, CEO,
Recruiter, Manager">
</div>

<!-- KEYWORDS INPUT -->
<div class="form-group">
  <label for="contact-keywords">Keywords (comma separated)</label>
  <input type="text" id="contact-keywords" placeholder="e.g., Machine
Learning, Fashion, E-commerce, Startup">
  <small>Pisahkan dengan koma untuk multiple keywords</small>

</div>
```

GANTI results-header:

html

```
<div class="results-header">
  <h2>Contact Results (<span id="contacts-count">0</span> found)</h2>
  <div class="result-actions">
    <!-- Bulk Actions -->
    <div class="bulk-actions">
      <button id="select-all-contacts" class="btn btn-sm">Select
All</button>
      <button id="deselect-all-contacts" class="btn btn-sm">Deselect
All</button>
      <button id="delete-selected-contacts" class="btn btn-sm
btn-danger">Delete Selected</button>
      <button id="edit-selected-contacts" class="btn btn-sm">Edit
Selected</button>
    </div>

    <!-- Email Actions -->
    <div class="email-actions">
      <button id="copy-emails-list" class="btn btn-sm
btn-success">Copy Emails</button>
      <button id="fast-blast-emails" class="btn btn-sm
btn-primary">Fast Blast</button>
```

```

        <button id="personalized-emails" class="btn btn-sm
btn-warning">Personalized Email</button>
        <button id="export-contacts" class="btn btn-sm
btn-success">Export CSV</button>
        <button id="clear-contacts" class="btn btn-sm btn-danger">Clear
All</button>
    </div>
</div>
</div>

<!-- Results Table -->
<div class="table-responsive">
    <table id="contacts-table" class="results-table">
        <thead>
            <tr>
                <th width="30"><input type="checkbox"
id="select-all-checkbox"></th>
                <th width="200">Email</th>
                <th width="100">Role</th>
                <th width="150">Source</th>
                <th width="150">Keyword</th>
                <th width="200">Overview</th>
                <th width="100">Actions</th>
            </tr>
        </thead>
        <tbody id="contacts-table-body">
            <!-- Dynamic rows will be inserted here -->
        </tbody>
    </table>
</div>

```

5.2 Step 2: Modifikasi contact_scraper.js

UPDATE startContactScraping function:

```

javascript
export async function startContactScraping(config) {
    console.log('startContactScraping called with config:', config);
    isContactScraping = true;
    contactOpenTabs = [];
    contactScrapingConfig = config;
    let allResults = [];

```

```

try {
  chrome.runtime.sendMessage({
    action: 'contactScrapingProgress',
    data: 0
  });

  // PARSE MULTIPLE KEYWORDS
  const keywordsArray = config.keywords
    ? config.keywords.split(',').map(k => k.trim()).filter(k => k)
    : [];

  const role = config.role || '';

  // Scrape LinkedIn dengan multiple keywords
  if (config.linkedinConfig && keywordsArray.length > 0) {
    console.log('Starting LinkedIn scraping for keywords:',
keywordsArray);

    for (const keyword of keywordsArray) {
      if (!isContactScraping) break;

      const linkedinResults = await scrapeBingForEmails(
        'linkedin',
        role ? `${role} ${keyword}` : keyword,
        config.linkedinConfig.dateRange,
        config.linkedinConfig.location,
        config.role || 'General'
      );
      allResults = allResults.concat(linkedinResults);

      // Update progress
      chrome.runtime.sendMessage({
        action: 'contactScrapingProgress',
        data: Math.min(100, (allResults.length / 100) * 100)
      });
    }
  }

  // Scrape Instagram dengan multiple keywords
  if (config.instagramConfig && keywordsArray.length > 0 &&
isContactScraping) {
    console.log('Starting Instagram scraping for keywords:',
keywordsArray);

    for (const keyword of keywordsArray) {

```

```

        if (!isContactScraping) break;

        const instagramResults = await scrapeBingForEmails(
            'instagram',
            role ? `${role} ${keyword}` : keyword,
            config.instagramConfig.dateRange,
            config.instagramConfig.location,
            config.role || 'General'
        );
        allResults = allResults.concat(instagramResults);
    }
}

if (isContactScraping) {
    console.log('Contact scraping completed with results:',
allResults.length);
    chrome.runtime.sendMessage({
        action: 'contactScrapingComplete',
        data: allResults
    });
}
} catch (error) {
    console.error('Contact scraping error:', error);
    chrome.runtime.sendMessage({
        action: 'contactScrapingError',
        data: error.message
    });
}

isContactScraping = false;
cleanupContactTabs();
}

```

UPDATE scrapeBingForEmails function dengan role:

```

javascript

export async function scrapeBingForEmails(source, keywords, dateRange,
location, role) {
    console.log(`Scraping ${source} with keywords: ${keywords}, role:
${role}`);

    // HARDCODED EMAIL PATTERNS
    const emailPatterns = [
        '@gmail.com',

```

```

    '"info@"',
    '"career@"',
    '"careers@"',
    '"hr@"',
    '"recruitment@"',
    '"contact@"',
    '"hello@"
  ];

  let allEmails = [];

  // Loop melalui semua email patterns
  for (const emailPattern of emailPatterns) {
    if (!isContactScraping) break;

    const baseQuery = `site:${source}.com ${keywords} ${emailPattern}`;
    let fullQuery = baseQuery;

    if (dateRange && dateRange !== 'custom') {
      const dateStr = getDateRange(dateRange);
      fullQuery += ` after:${dateStr.start} before:${dateStr.end}`;
    } else if (dateRange === 'custom' && contactScrapingConfig) {
      fullQuery += `
after:${contactScrapingConfig.linkedinConfig.dateRange.start}
before:${contactScrapingConfig.linkedinConfig.dateRange.end}`;
    }

    if (location) {
      fullQuery += ` location:${location}`;
    }

    console.log(`Query for ${emailPattern}:`, fullQuery);

    try {
      const { emails } = await scrapeBingPage(fullQuery, 1, source,
keywords, role);
      allEmails = allEmails.concat(emails);
    } catch (error) {
      console.error(`Error scraping with pattern ${emailPattern}:`,
error);
    }
  }

  // Remove duplicates
  const uniqueEmails = [];
  const emailSet = new Set();

```

```

    allEmails.forEach(emailObj => {
      if (!emailSet.has(emailObj.email)) {
        emailSet.add(emailObj.email);
        uniqueEmails.push(emailObj);
      }
    });

    console.log(`Total unique emails found for ${source}:
    ${uniqueEmails.length}`);
    return uniqueEmails;
  }
}

```

UPDATE scrapeBingPage function:

```

javascript

export async function scrapeBingPage(query, page, source, keyword, role) {
  console.log(`Opening Google page ${page} for query: ${query}`);
  return new Promise((resolve, reject) => {
    const url =
`https://www.google.com/search?q=${encodeURIComponent(query)}`;

    chrome.tabs.create({ url, active: false, pinned: true }, (tab) => {
      if (chrome.runtime.lastError) {
        return reject(new Error(chrome.runtime.lastError.message));
      }

      const tabId = tab.id;
      contactOpenTabs.push(tabId);

      let hasProcessed = false;
      let collectedEmails = [];

      const scrapeLoop = (attempt = 1) => {
        if (!isContactScraping || attempt > 5) {
          cleanupContactTab(tabId);
          resolve({ emails: collectedEmails, pageContent: '' });
          return;
        }
      }

      chrome.scripting.executeScript({
        target: { tabId },
        func: async () => {
          // Scroll down gradually

```

```

for (let i = 0; i < 5; i++) {
  window.scrollBy(0, document.body.scrollHeight);
  await new Promise(r => setTimeout(r, 1000));
}

// ENHANCED EMAIL EXTRACTION WITH MULTIPLE PATTERNS
const pageText = document.body.innerText;
const emailRegexes = [
  /[a-zA-Z0-9._%+-]+@gmail\.com/gi,
  /info@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,
  /career(?:s)?@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,
  /hr@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,
  /recruitment@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,
  /contact@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi,

  /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/gi
];

let allMatches = [];
emailRegexes.forEach(regex => {
  const matches = pageText.match(regex) || [];
  allMatches = allMatches.concat(matches);
});

const matches = [...new Set(allMatches)];

// Try to find and click the "Next" button
const nextBtn = [...document.querySelectorAll('a')]
  .find(a =>
a.innerText.toLowerCase().includes('next'));

if (nextBtn) {
  nextBtn.click();
  return {
    emails: matches,
    hasNext: true,
    pageText: pageText
  };
} else {
  return {
    emails: matches,
    hasNext: false,
    pageText: pageText
  };
}
}

```

```

    }, (results) => {
      if (chrome.runtime.lastError) {
        cleanupContactTab(tabId);
        reject(new
Error(chrome.runtime.lastError.message));
        return;
      }

      const result = results?.[0]?.result;
      if (!result) {
        cleanupContactTab(tabId);
        reject(new Error('No result returned'));
        return;
      }

      // Process emails with context
      const newEmails = result.emails.map(email => {
        const emailIndex = result.pageText.indexOf(email);
        let context = `Found on ${source} search`;

        if (emailIndex !== -1) {
          const start = Math.max(0, emailIndex - 100);
          const end = Math.min(result.pageText.length,
emailIndex + email.length + 100);
          context = result.pageText.substring(start,
end).replace(/\s+/g, ' ').trim();
        }

        return {
          email: email.toLowerCase(),
          overview: context,
          source: source.charAt(0).toUpperCase() +
source.slice(1),

          keyword: keyword,
          role: role || 'General',
          selected: false
        };
      });

      // Filter duplicates
      const uniqueNewEmails = newEmails.filter(newEmail =>
        !collectedEmails.some(existing =>
          existing.email.toLowerCase() ===
newEmail.email.toLowerCase()
        )
      );
    });

```

```

        collectedEmails.push(...uniqueNewEmails);

        console.log(`Collected ${collectedEmails.length} so far
(attempt ${attempt})`);

        if (result.hasNext && attempt < 10) {
            setTimeout(() => scrapeLoop(attempt + 1), 4000 +
Math.random() * 2000);
        } else {
            cleanupContactTab(tabId);
            resolve({ emails: collectedEmails, pageContent: ''
});
        }
    });
};

// Wait for first page load
const onTabUpdated = (updatedTabId, changeInfo) => {
    if (updatedTabId === tabId && changeInfo.status ===
'complete' && !hasProcessed) {
        hasProcessed = true;
        chrome.tabs.onUpdated.removeListener(onTabUpdated);
        setTimeout(() => scrapeLoop(), 3000);
    }
};
chrome.tabs.onUpdated.addListener(onTabUpdated);
});
});
}

```

TAMBAHKAN fungsi-fungsi baru di akhir file:

```
javascript
```

```
// ===== NEW FUNCTIONS =====
```

```
// Email Actions Functions
```

```
export function copyEmailList(emails) {
    const emailList = emails.map(e => e.email).join(', ');
    navigator.clipboard.writeText(emailList).then(() => {
        console.log('Emails copied to clipboard');
        chrome.runtime.sendMessage({
            action: 'showNotification',
            data: {

```

```

        title: 'Success',
        message: `${emails.length} emails copied to clipboard`
    }
    });
}).catch(err => {
    console.error('Failed to copy emails:', err);
});
}

export function openFastBlast(emails, subject = '', body = '') {
    const emailList = emails.map(e => e.email).join(',');
    const gmailUrl =
`https://mail.google.com/mail/?view=cm&fs=1&bcc=${emailList}&su=${encodeURIComponent(
Component(subject))}&body=${encodeURIComponent(body)}`;

    chrome.tabs.create({ url: gmailUrl, active: true }, (tab) => {
        console.log('Opened Gmail compose for fast blast');
    });
}

export function openPersonalizedEmailModal(emails) {
    // Ini akan dipanggil dari popup.js untuk membuka modal personalized
    email
    chrome.runtime.sendMessage({
        action: 'openPersonalizedEmailModal',
        data: { emails: emails }
    });
}

// Results Management Functions
export function selectAllContacts() {
    chrome.runtime.sendMessage({
        action: 'selectAllContacts'
    });
}

export function deselectAllContacts() {
    chrome.runtime.sendMessage({
        action: 'deselectAllContacts'
    });
}

export function deleteSelectedContacts(contactIds) {
    chrome.runtime.sendMessage({
        action: 'deleteSelectedContacts',
        data: { ids: contactIds }
    });
}

```

```

    });
}

export function editContact(contactId, updates) {
  chrome.runtime.sendMessage({
    action: 'editContact',
    data: { id: contactId, updates: updates }
  });
}

// Export to CSV
export function exportToCSV(contacts) {
  const headers = ['Email', 'Role', 'Source', 'Keyword', 'Overview'];
  const csvRows = [];

  // Add headers
  csvRows.push(headers.join(','));

  // Add data rows
  contacts.forEach(contact => {
    const row = [
      `${contact.email}`,
      `${contact.role}`,
      `${contact.source}`,
      `${contact.keyword}`,
      `${contact.overview.replace(/"/g, '""')}`
    ];
    csvRows.push(row.join(','));
  });

  const csvString = csvRows.join('\n');
  const blob = new Blob([csvString], { type: 'text/csv' });
  const url = window.URL.createObjectURL(blob);

  chrome.downloads.download({
    url: url,
    filename: `contacts_export_${new
Date().toISOString().split('T')[0]}.csv`
  });
}

// Email Validation
export function validateEmail(email) {
  const re = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
  return re.test(email);
}

```

```

// Bulk Edit Function
export function bulkEditContacts(contactIds, field, value) {
  chrome.runtime.sendMessage({
    action: 'bulkEditContacts',
    data: { ids: contactIds, field: field, value: value }
  });
}

// Progress Tracking
let scrapingProgress = {
  totalKeywords: 0,
  completedKeywords: 0,
  totalEmails: 0,
  currentSource: ''
};

export function updateProgress(keywordCompleted = false) {
  if (keywordCompleted) {
    scrapingProgress.completedKeywords++;
  }

  const progress = Math.floor(
    (scrapingProgress.completedKeywords /
scrapingProgress.totalKeywords) * 100
  );

  chrome.runtime.sendMessage({
    action: 'contactScrapingProgress',
    data: progress
  });
}

// Rate Limiting to avoid blocks
const rateLimiter = {
  lastRequest: 0,
  minDelay: 3000, // 3 seconds between requests

  async wait() {
    const now = Date.now();
    const timeSinceLast = now - this.lastRequest;

    if (timeSinceLast < this.minDelay) {
      await new Promise(resolve =>
        setTimeout(resolve, this.minDelay - timeSinceLast)
      );
    }
  }
};

```

```

    }

    this.lastRequest = Date.now();
  }
};

// LinkedIn Direct Scraping Helper (Future Enhancement)
export async function scrapeLinkedInDirect(keywords, role, location) {
  // Placeholder for direct LinkedIn scraping
  // This would require LinkedIn API or careful web scraping
  console.log('Direct LinkedIn scraping not yet implemented');
  return [];
}

```

5.3 Step 3: Tambahkan CSS Styling (di style.css yang ada)

```

css

/* ===== ENHANCED RESULTS TABLE ===== */
.results-table-container {
  margin-top: 20px;
  overflow-x: auto;
}

.results-table {
  width: 100%;
  border-collapse: collapse;
  background: white;
  border-radius: 8px;
  overflow: hidden;
  box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.results-table th {
  background: #f8f9fa;
  padding: 12px 15px;
  text-align: left;
  font-weight: 600;
  color: #495057;
  border-bottom: 2px solid #dee2e6;
}

.results-table td {
  padding: 10px 15px;
}

```

```
        border-bottom: 1px solid #dee2e6;
        vertical-align: middle;
    }

.results-table tbody tr:hover {
    background-color: #f8f9fa;
}

.results-table input[type="checkbox"] {
    margin: 0;
    cursor: pointer;
}

.results-table .email-cell {
    font-family: monospace;
    color: #007bff;
}

.results-table .action-cell {
    display: flex;
    gap: 5px;
}

.results-table .btn-sm {
    padding: 3px 8px;
    font-size: 12px;
}

/* ===== BULK ACTIONS ===== */
.bulk-actions {
    display: flex;
    gap: 8px;
    margin-bottom: 10px;
    flex-wrap: wrap;
}

.email-actions {
    display: flex;
    gap: 8px;
    margin-top: 10px;
    flex-wrap: wrap;
}

.result-actions {
    display: flex;
    flex-direction: column;
}
```

```

        gap: 10px;
        margin: 15px 0;
    }

    /* ===== FORM ENHANCEMENTS ===== */
    .form-group {
        margin-bottom: 15px;
    }

    .form-group label {
        display: block;
        margin-bottom: 5px;
        font-weight: 500;
    }

    .form-group input[type="text"],
    .form-group select {
        width: 100%;
        padding: 8px 12px;
        border: 1px solid #ced4da;
        border-radius: 4px;
        font-size: 14px;
    }

    .form-group small {
        display: block;
        margin-top: 5px;
        color: #6c757d;
        font-size: 12px;
    }

    /* ===== STATUS INDICATORS ===== */
    .status-indicator {
        display: inline-block;
        width: 10px;
        height: 10px;
        border-radius: 50%;
        margin-right: 5px;
    }

    .status-active {
        background-color: #28a745;
    }

    .status-inactive {
        background-color: #dc3545;
    }

```

```

}

.status-pending {
    background-color: #ffc107;
}

/* ===== MODAL STYLES ===== */
.modal-overlay {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background: rgba(0,0,0,0.5);
    display: none;
    z-index: 1000;
}

.modal-content {
    background: white;
    border-radius: 8px;
    padding: 20px;
    max-width: 500px;
    margin: 50px auto;
    max-height: 80vh;
    overflow-y: auto;
}

.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 20px;
    padding-bottom: 10px;
    border-bottom: 1px solid #dee2e6;
}

.modal-close {
    background: none;
    border: none;
    font-size: 24px;
    cursor: pointer;
    color: #6c757d;
}

/* ===== RESPONSIVE DESIGN ===== */

```

```
@media (max-width: 768px) {  
  .results-table th,  
  .results-table td {  
    padding: 8px 10px;  
    font-size: 13px;  
  }  
  
  .bulk-actions,  
  .email-actions {  
    flex-direction: column;  
  }  
  
  .bulk-actions button,  
  .email-actions button {  
    width: 100%;  
    margin-bottom: 5px;  
  }  
}
```

6. TESTING CHECKLIST

6.1 Functional Testing

- Input role field muncul dan berfungsi
- Multiple keywords dipisahkan koma berhasil diparse
- Email patterns (info@, careers@, hr@) terdeteksi
- Checkbox selection bekerja (select all, deselect all)
- Delete selected contacts bekerja
- Copy email list ke clipboard
- Fast Blast buka Gmail dengan BCC
- Export CSV menghasilkan file yang benar

6.2 Integration Testing

- Scraping dengan role + multiple keywords
- Progress bar update selama scraping
- Results display dengan role column
- Bulk edit multiple contacts
- Storage persistency saat tab ditutup

6.3 Error Handling

- CAPTCHA detection masih bekerja
- Rate limiting mencegah block
- Error messages muncul dengan jelas
- Stop scraping berfungsi saat proses berjalan

7. DEPLOYMENT STEPS

7.1 Update Manifest Permissions

```
json
{
  "permissions": [
    "storage",
    "activeTab",
    "scripting",
    "downloads",
    "clipboardWrite",
    "notifications"
  ]
}
```

7.2 Pack Extension

1. Di `chrome://extensions/`, klik "Pack extension"
2. Pilih folder `ExtensionSwakarsa`
3. Klik "Pack Extension"
4. File `.crx` dan `.pem` akan dihasilkan

7.3 Distribution Options

- Developer Mode: Load unpacked (selama development)
- Chrome Web Store: Upload packaged `.crx`
- Private Distribution: Host `.crx` file sendiri

8. TROUBLESHOOTING COMMON ISSUES

Issue 1: Extension tidak load

text

Solution:

1. Pastikan semua file ada di folder yang benar
2. Check console errors di chrome://extensions/
3. Reload extension

Issue 2: Scraping tidak menemukan email

text

Solution:

1. Check Google query di console log
2. Verifikasi keywords dan role format
3. Test regex patterns manual
4. Check CAPTCHA detection

Issue 3: Clipboard tidak bekerja

text

Solution:

1. Pastikan permission clipboardWrite ada di manifest
2. Check console untuk permission errors
3. Test di HTTPS site (clipboard requires secure context)

Issue 4: Gmail deeplink tidak terbuka

text

Solution:

1. Verify email format (comma separated, no spaces)
2. Check URL encoding
3. Test dengan jumlah email kecil terlebih dahulu

9. NEXT STEPS UNTUK ENHANCEMENT

Phase 2 (Optional):

1. Implementasi direct LinkedIn scraping tanpa Google
2. Google Apps Script integration untuk personalized email
3. Template editor dengan variables
4. Scheduled email sending
5. Analytics dashboard

10. CONTACT & SUPPORT

Jika ada masalah saat implementasi:

1. Check GitHub issues:
<https://github.com/Rmyonathan/ExtensionSwakarsa/issues>
2. Review console errors di DevTools (F12)
3. Test dengan sample data kecil terlebih dahulu
4. Pastikan Chrome versi terbaru

NOTE: Implementasi ini menjaga backward compatibility dengan kode existing. Semua perubahan incremental dan tidak menghapus fungsi yang sudah ada.